

Universidad de Unisabaneta

Facultad de Ingeniería

Calidad de Software

Sistema de Gestión y Migración de la Biblioteca Central

Documentación Técnica del Pipeline de Migración

Autor: Juan José Barrera Cardona

Docente: Manolo Pajaro

Mayo 2026

1. Descripción General del Proyecto

El Sistema de Gestión y Migración de la Biblioteca Central es un pipeline de datos desarrollado en Python cuyo objetivo es transformar una base de datos legada no normalizada (PostgreSQL) hacia una arquitectura relacional normalizada y posteriormente migrarla a una base de datos documental en MongoDB.

El sistema abarca tres fases principales: poblamiento y validación de datos en el esquema legado, migración ELT hacia el esquema normalizado con ejecución de queries analíticos avanzados, y finalmente la migración hacia MongoDB con estructura embebida orientada a documentos.

1.1 Objetivos

- Poblar una base de datos legada con datos sintéticos representativos.
- Validar la calidad de los datos en ambos contextos (legacy y normalizado).
- Ejecutar la migración ELT desde el esquema legado al esquema normalizado.
- Ejecutar queries analíticos sobre los datos migrados (vistas, funciones, procedimientos).
- Migrar los datos normalizados a MongoDB con estructura documental embebida.

1.2 Tecnologías Utilizadas

Función	Descripción	Base de Datos
Python 3.12	Lenguaje principal del pipeline	—
PostgreSQL	Base de datos relacional (legacy y normalizada)	biblioteca_legacy / biblioteca_normalizada
MongoDB	Base de datos documental destino	biblioteca_mongo
psycopg2	Conector Python-PostgreSQL	—
pymongo	Conector Python-MongoDB	—
Faker	Generación de datos sintéticos	—

2. Arquitectura y Bases de Datos

2.1 Esquema Legado — biblioteca_legacy

La base de datos legada representa un diseño no normalizado con columnas combinadas, datos multivalor y tipos inconsistentes. Contiene cuatro tablas principales:

Función	Descripción	Base de Datos
Biblioteca_Data	Almacena libros con columna combinada categoria_y_descripcion	biblioteca_legacy
Prestamos_Crudos	Préstamos con libros múltiples en una sola columna de texto	biblioteca_legacy
Inventario_Sedes	Inventario de libros por sede sin relaciones formales	biblioteca_legacy
Reseñas_Usuarios	Reseñas con calificación almacenada como texto	biblioteca_legacy

2.2 Esquema Normalizado — biblioteca_normalizada

El esquema normalizado sigue la Tercera Forma Normal (3FN) con relaciones explícitas:

Función	Descripción	Base de Datos
Libros	Datos del libro (título, fecha, editorial FK)	biblioteca_normalizada
Autores	Catálogo de autores	biblioteca_normalizada
Libro_Autor	Relación N:M entre Libros y Autores	biblioteca_normalizada
Categorias	Catálogo de categorías con descripción separada	biblioteca_normalizada
Libro_Categoria	Relación N:M entre Libros y Categorías	biblioteca_normalizada
Editoriales	Catálogo de editoriales	biblioteca_normalizada
Usuarios	Usuarios del sistema con nombre y correo	biblioteca_normalizada
Prestamos	Cabecera del préstamo con estado y fecha de salida	biblioteca_normalizada
Prestamo_Libro	Relación N:M entre Préstamos y Libros	biblioteca_normalizada
Sedes	Sedes con nombre y ubicación	biblioteca_normalizada
Inventario	Stock por sede y libro	biblioteca_normalizada
Reseñas	Reseñas con calificación numérica y FK validadas	biblioteca_normalizada
Log_Prestamos	Log generado por el procedimiento almacenado	biblioteca_normalizada

2.3 Esquema Documental — biblioteca_mongo

MongoDB almacena los datos con estructura embebida, agrupando entidades relacionadas en documentos únicos:

Función	Descripción	Base de Datos
libros	Documento con editorial, autores y categorías embebidos	biblioteca_mongo
usuarios	Documento plano con nombre y correo	biblioteca_mongo
prestamos	Documento con usuario embebido y array de libros prestados	biblioteca_mongo
inventario	Documento con sede y referencia al libro embebidos	biblioteca_mongo

3. Guía de Instalación y Uso

3.1 Requisitos Previos

- Python 3.12 o superior
- PostgreSQL 14 o superior con dos bases de datos creadas: biblioteca_legacy y biblioteca_normalizada
- MongoDB 6 o superior ejecutándose en localhost:27017
- Librerías Python: psycopg2, pymongo, Faker

3.2 Instalación de Dependencias

```
pip install psycopg2-binary pymongo faker
```

3.3 Estructura de Carpetas Requerida

```
proyecto/
├── main.py
├── config/
│   ├── config_calidad.json
│   └── mapping.json
├── sql/
│   ├── BD_legacy.sql
│   ├── BD_norm.sql
│   ├── SP1.sql
│   ├── SP2.sql
│   ├── SP3.sql
│   ├── SP4.sql
│   ├── Trigger_control_inv.sql
│   ├── migracion_elt.sql
│   ├── vista_libros_mas_prestados.sql
│   ├── calcular_multa.sql
│   ├── generar_log_prestamos.sql
│   └── Log_Prestamos.sql
└── logs/
    └── errores.log (generado automáticamente)
```

3.4 Configuración de Conexión

Las credenciales de conexión se configuran directamente en las funciones del main.py:

```
host      = 'localhost'
database  = 'biblioteca_legacy' | 'biblioteca_normalizada'
user      = 'postgres'
password  = '<tu_contraseña>'
```

3.5 Ejecución

```
python main.py
```

El pipeline se ejecuta de forma secuencial y automática. No requiere argumentos adicionales.

4. Detalle de Funciones del main.py

4.1 conectar()

Establece y retorna una conexión a la base de datos biblioteca_legacy.

- Parámetros: ninguno
- Retorna: objeto de conexión psycopg2
- Base de datos: biblioteca_legacy

```
conn = conectar()
```

4.2 limpiar_tablas(conn)

Trunca las cuatro tablas legadas con RESTART IDENTITY CASCADE, dejando el esquema limpio antes del poblamiento.

- Parámetros: conn — conexión activa a biblioteca_legacy
- Retorna: None
- Efecto: elimina todos los registros y reinicia los contadores de secuencia

4.3 poblar_datos(conn)

Genera e inserta 250 registros sintéticos en las cuatro tablas legadas utilizando la librería Faker.

- Parámetros: conn — conexión activa a biblioteca_legacy
- Retorna: None
- Genera datos para: Biblioteca_Data, Prestamos_Crudos, Reseñas_Usuarios, Inventario_Sedes
- Nota: los datos incluyen intencionalmente anomalías (campos combinados, multivalor) propias del esquema legado

4.4 validar_datos(conn, contexto)

Lee las reglas de validación del archivo config/config_calidad.json y las aplica sobre las tablas correspondientes al contexto indicado. Genera un reporte en logs/errores.log.

- Parámetros: conn — conexión activa | contexto — 'legacy' o 'normalizada'
- Retorna: None
- Tipos de validación soportados: regex, rango_numerico, multivalor, campo_combinado, formato_fecha
- Salida: archivo logs/errores.log con conteo de errores por tabla y columna

4.5 ejecutar_migracion_elt()

Abre y ejecuta el script `sql/migracion_elt.sql` sobre `biblioteca_normalizada`. Este script implementa el proceso ELT (Extract-Load-Transform) que transforma los datos legados al esquema normalizado.

- Parámetros: ninguno
- Retorna: None
- Base de datos destino: `biblioteca_normalizada`
- Archivo SQL: `sql/migracion_elt.sql`

4.6 validar_normalizada()

Abre una conexión a `biblioteca_normalizada` e invoca `validar_datos()` con contexto `'normalizada'`.

- Parámetros: ninguno
- Retorna: None
- Complementa el log generado en la validación legacy (modo append)

4.7 consulta_libros_mas_prestados()

Crea o reemplaza la vista `vista_libros_mas_prestados` en `biblioteca_normalizada` y consulta los primeros 50 registros ordenados por frecuencia de préstamo descendente.

- Parámetros: ninguno
- Retorna: lista de tuplas (titulo, usuario, veces_prestado)
- Archivo SQL: `sql/vista_libros_mas_prestados.sql`
- Separación de statements: `split(';')` — CREATE VIEW y SELECT ejecutados por separado
- Salida en terminal: tabla formateada con los 50 libros más prestados

4.8 consulta_top_multas()

Crea o reemplaza la función PL/pgSQL `calcular_multa()` y ejecuta un SELECT que retorna los 10 usuarios con mayor multa acumulada por retraso en préstamos activos. La multa se calcula como: $(\text{días desde fecha_salida} - 7) \times \1.000 por día.

- Parámetros: ninguno
- Retorna: lista de tuplas (nombre, correo, id_prestamo, fecha_salida, dias_retraso, multa_total)
- Archivo SQL: `sql/calcular_multa.sql`
- Separación de statements: `split('-- RUN_QUERY')` — necesario para no romper el cuerpo \$\$ de la función PL/pgSQL
- Filtro: solo préstamos con estado = 'Activo' y retraso mayor a 7 días
- Salida en terminal: tabla con top 10 usuarios y monto de multa formateado

4.9 ejecutar_log_prestamos()

Crea o reemplaza el procedimiento almacenado `generar_log_prestamos()` y lo invoca mediante `CALL`. El procedimiento usa un cursor explícito para recorrer los préstamos activos e insertar cada registro en la tabla `Log_Prestamos`.

- Parámetros: ninguno
- Retorna: None
- Archivo SQL: `sql/generar_log_prestamos.sql`
- Separación de statements: `split('-- RUN_QUERY')` — mismo patrón que `calcular_multa`
- Salida en terminal: confirmación de creación del procedimiento y ejecución del log (sin datos)
- Nota: requiere que la tabla `Log_Prestamos` exista en `biblioteca_normalizada`

4.10 limpiar_mongo()

Conecta a MongoDB y elimina (drop) todas las colecciones existentes en `biblioteca_mongo`.

- Parámetros: ninguno
- Retorna: None
- Base de datos: `biblioteca_mongo` en `localhost:27017`

4.11 migrar_a_mongo()

Migra los datos desde `biblioteca_normalizada` (PostgreSQL) hacia `biblioteca_mongo` (MongoDB) construyendo documentos embebidos para cada colección.

- Parámetros: ninguno
- Retorna: None
- Colecciones generadas: libros (con autores, categorías y editorial embebidos), usuarios, prestamos (con usuario y libros embebidos), inventario (con sede y libro embebidos)
- Nota: convierte fechas `DATE` de PostgreSQL a objetos `datetime` de Python para compatibilidad con MongoDB

5. Flujo Completo del Pipeline

El pipeline se ejecuta de forma secuencial desde la función `main()`. Cada etapa depende de la correcta finalización de la anterior.

Paso	Función	BD	Descripción
1	<code>conectar()</code>	<code>biblioteca_legacy</code>	Conexión inicial a PostgreSQL legado
2	<code>limpiar_tablas()</code>	<code>biblioteca_legacy</code>	Trunca las 4 tablas legadas
3	<code>poblar_datos()</code>	<code>biblioteca_legacy</code>	Inserta 250 registros sintéticos
4	<code>validar_datos(..., 'legacy')</code>	<code>biblioteca_legacy</code>	Valida calidad del esquema legado
5	<code>ejecutar_migracion_elt()</code>	<code>biblioteca_normalizada</code>	Ejecuta <code>migracion_elt.sql</code> (ELT)
6	<code>validar_normalizada()</code>	<code>biblioteca_normalizada</code>	Valida calidad del esquema normalizado
7	<code>consulta_libros_mas_prestados()</code>	<code>biblioteca_normalizada</code>	Crea vista y muestra top 50
8	<code>consulta_top_multas()</code>	<code>biblioteca_normalizada</code>	Crea función y muestra top 10 multas
9	<code>ejecutar_log_prestamos()</code>	<code>biblioteca_normalizada</code>	Crea procedimiento y genera log
10	<code>limpiar_mongo()</code>	<code>biblioteca_mongo</code>	Limpia colecciones MongoDB
11	<code>migrar_a_mongo()</code>	<code>biblioteca_mongo</code>	Migra datos con documentos embebidos
12	Verificación final	<code>biblioteca_mongo</code>	Cuenta documentos por colección

5.1 Notas sobre la Ejecución

- El pipeline maneja excepciones de forma independiente por función: un error en una etapa no detiene las siguientes salvo que sea en `main()` directamente.
- Los archivos SQL se leen desde la carpeta `sql/` usando rutas absolutas calculadas con `os.path.dirname(os.path.abspath(__file__))`, lo que garantiza portabilidad entre sistemas operativos.
- Las funciones y procedimientos PL/pgSQL utilizan el separador `-- RUN_QUERY` en el archivo `.sql` para evitar que el `split(';')` rompa el cuerpo delimitado por `$$`.
- El log de validación se escribe en modo `'w'` para el contexto `legacy` y en modo `'a'` para el contexto `normalizado`, consolidando ambas validaciones en un único archivo `logs/errores.log`.

- Las fechas de PostgreSQL (tipo DATE) se convierten a datetime de Python usando `datetime.combine(fecha, datetime.min.time())` antes de insertarlas en MongoDB.

6. Archivos SQL Independientes

A diferencia de los archivos SQL invocados automáticamente por el pipeline desde main.py, los archivos descritos en esta sección se ejecutan de forma manual directamente sobre PostgreSQL (mediante pgAdmin, psql u otro cliente SQL). Su propósito es la definición de esquemas, la gestión operativa de la biblioteca y el control de integridad del inventario.

6.1 BD_legacy.sql — Esquema de la Base de Datos Legada

Define el esquema inicial no normalizado de biblioteca_legacy. Contiene las cuatro tablas con las anomalías de diseño intencionales que sirven como punto de partida del proceso de migración.

Función	Descripción	Base de Datos
Biblioteca_Data	Campo combinado categoria_y_descripcion (viola 1FN) y fecha como VARCHAR	biblioteca_legacy
Prestamos_Crudos	Lista multivalorada de libros en una sola columna de texto (viola 1FN)	biblioteca_legacy
Inventario_Sedes	Sin claves foraneas explícitas, datos de sede y libro redundantes	biblioteca_legacy
Resenhas_Usuarios	Calificacion almacenada como VARCHAR en lugar de tipo numérico	biblioteca_legacy

Este archivo se ejecuta una única vez para crear el esquema legado antes de iniciar el pipeline. Las anomalías documentadas en los comentarios del SQL son objeto de estudio del proceso de calidad.

6.2 BD_norm.sql — Esquema de la Base de Datos Normalizada

Define el esquema normalizado de biblioteca_normalizada siguiendo la Tercera Forma Normal (3FN). Separa todas las entidades combinadas del esquema legado en tablas independientes con relaciones explícitas mediante claves foráneas.

Función	Descripción	Base de Datos
Editoriales	Catalogo independiente de editoriales	biblioteca_normalizada
Libros	Entidad libro con FK a Editoriales y fecha como tipo DATE	biblioteca_normalizada
Autores	Catalogo independiente de autores	biblioteca_normalizada
Libro_Autor	Tabla puente N:M — resuelve la relación libro-autor	biblioteca_normalizada
Categorias	Categoría y descripción separadas en columnas	biblioteca_normalizada

	independientes	
Libro_Categoria	Tabla puente N:M — resuelve la relación libro-categoría	biblioteca_normalizada
Usuarios	Entidad usuario con nombre y correo	biblioteca_normalizada
Prestamos	Cabecera del préstamo con FK a Usuarios y estado controlado	biblioteca_normalizada
Prestamo_Libro	Tabla puente N:M — resuelve la lista multivalorada del legado	biblioteca_normalizada
Sedes	Catalogo independiente de sedes con ubicación	biblioteca_normalizada
Inventario	Stock por sede y libro con PK compuesta (id_sede, id_libro)	biblioteca_normalizada
Resenhas	Calificacion como INT con FK a Usuarios y Libros	biblioteca_normalizada

Este archivo se ejecuta una única vez antes de iniciar el pipeline para preparar el esquema destino de la migración ELT.

6.3 SP1.sql — Procedimiento: registrar_prestamo()

Registra un nuevo préstamo en el sistema a partir del correo del usuario, el ID del libro y el estado inicial. Busca el usuario por correo, crea el registro en Prestamos y genera la relación en Prestamo_Libro en una sola transacción.

- Parámetros: p_correo TEXT, p_id_libro INT, p_estado TEXT
- Valida que el usuario exista; lanza excepción si no se encuentra
- Uso de ejemplo: CALL registrar_prestamo('correo@ejemplo.com', 1, 'Activo')
- Tablas afectadas: Prestamos, Prestamo_Libro

Nota: al insertar en Prestamo_Libro se activa automáticamente el trigger trg_controlar_inventario, que verifica stock disponible y descuenta una unidad del inventario.

6.4 SP2.sql — Procedimiento: devolver_libro()

Marca un préstamo existente como devuelto actualizando el campo estado a Devuelto. Lanza una excepción si el ID de préstamo no existe en la tabla.

- Parámetros: p_id_prestamo INT
- Valida existencia del préstamo con IF NOT FOUND
- Uso de ejemplo: CALL devolver_libro(1)
- Tablas afectadas: Prestamos

Nota: actualmente la devolución no incrementa el stock de vuelta en Inventario. Este comportamiento puede extenderse con un trigger ON UPDATE sobre Prestamos.

6.5 SP3.sql — Procedimiento: registrar_resena()

Inserta una reseña de un usuario sobre un libro con validación de rango en la calificación. Garantiza que solo se registren calificaciones entre 1 y 5.

- Parámetros: p_id_usuario INT, p_id_libro INT, p_comentario TEXT, p_calificacion INT
- Valida que p_calificacion esté entre 1 y 5; lanza excepción si está fuera de rango
- Uso de ejemplo: CALL registrar_resena(1, 2, 'Muy buen libro', 5)
- Tablas afectadas: Reseñas

6.6 SP4.sql — Función: obtener_libros_por_categoria()

Función de tabla (RETURNS TABLE) que retorna todos los libros pertenecientes a una categoría específica, realizando los JOINS necesarios a través de la tabla puente Libro_Categoria.

- Parámetros: p_categoria TEXT
- Retorna: tabla con columnas titulo (TEXT) y categoria (TEXT)
- Uso de ejemplo: SELECT * FROM obtener_libros_por_categoria('Terror')
- Tablas consultadas: Libros, Libro_Categoria, Categorías

6.7 Trigger_control_inv.sql — Trigger de Control de Inventario

Implementa el control automático de stock mediante una función trigger y su asociación a la tabla Prestamo_Libro. Se compone de dos partes: la función fn_controlar_inventario() y el trigger trg_controlar_inventario.

Función: fn_controlar_inventario()

Se ejecuta automáticamente antes de cada INSERT en Prestamo_Libro. Su lógica es:

- Consulta el stock del libro en Inventario usando NEW.id_libro
- Lanza excepción si el libro no existe en inventario
- Lanza excepción si el stock es igual o menor a cero
- Descuenta una unidad del stock si la validación pasa
- Retorna NEW para permitir que el INSERT proceda

Trigger: trg_controlar_inventario

- Tipo: BEFORE INSERT — se ejecuta antes de que el registro entre a Prestamo_Libro
- Granularidad: FOR EACH ROW — se evalúa individualmente por cada libro del préstamo
- Tabla asociada: Prestamo_Libro

Comportamiento durante la Migración ELT

Durante la ejecución de migracion_elt.sql el trigger es desactivado temporalmente para permitir la carga masiva de datos históricos sin restricciones de stock. Una vez finalizada la migración, el trigger es reactivado para retomar el control operativo del inventario.

```
-- Desactivar durante la migración
ALTER TABLE Prestamo_Libro DISABLE TRIGGER trg_controlar_inventario;

-- ... carga de datos ELT ...

-- Reactivar al finalizar
ALTER TABLE Prestamo_Libro ENABLE TRIGGER trg_controlar_inventario;
```

Este patrón es una práctica estándar en procesos ELT para evitar que los triggers de negocio interfieran con cargas masivas de datos históricos donde el stock no refleja el estado real durante la carga (en este caso el proceso se automatizo).

6.8 Log_Prestamos.sql — Tabla de Log de Préstamos

Define la tabla Log_Prestamos en biblioteca_normalizada. Esta tabla es el destino de los registros generados por el procedimiento generar_log_prestamos() y debe crearse manualmente antes de ejecutar dicho procedimiento.

Función	Descripción	Base de Datos
id_log	SERIAL PRIMARY KEY — identificador autoincrementable del registro de log	biblioteca_normalizada
id_prestamo	INT — referencia al préstamo registrado	biblioteca_normalizada
usuario	TEXT — nombre del usuario al momento del log	biblioteca_normalizada
fecha_salida	DATE — fecha de salida del préstamo original	biblioteca_normalizada
fecha_log	TIMESTAMP DEFAULT CURRENT_TIMESTAMP — momento exacto del registro	biblioteca_normalizada

Usa IF NOT EXISTS para que el script sea idempotente: puede ejecutarse múltiples veces sin generar errores si la tabla ya existe. Una vez creada, el procedimiento generar_log_prestamos() puede invocarse desde el pipeline (main.py) o manualmente mediante CALL generar_log_prestamos().